

ISM3232 — Week 1 Lab

Developer Mindset & First Setup — Instructor Facilitation Guide

ISM3232 · Business Application Development · USF Muma College of Business

Module 01 · Unit 1 · Developer Foundations

Contents

1	Session Snapshot	2
2	Learning Objectives	2
3	Before Class — Setup Checklist	3
4	Materials Needed	3
5	Timing Plan (75 minutes)	3
6	Segment-by-Segment Walkthrough	5
6.1	Intro (0:00–0:05)	5
6.2	Part 1 — Verify Your Environment (0:05–0:18, 13 min)	5
6.3	Part 2 — Course Folder Structure (0:18–0:35, 17 min)	7
6.4	Part 3 — First Python Script (0:35–0:58, 23 min)	9
6.5	Part 4 — README (0:58–1:10, 12 min)	12
6.6	Stretch — Configure Git Identity (1:10–1:15, as time allows)	14
7	Wrap-Up (last ~5 minutes)	15
8	Appendix A — Full Command & Code Reference	15
9	Appendix B — Extra Practice (only if the class finishes early)	16

1 Session Snapshot

Course	ISM3232 — Business Application Development
Session	Week 1 Lab — Developer Mindset & First Setup
Unit	Unit 1 · Developer Foundations
Class length	Full class period (75 minutes)
Format	Live terminal code-along — every student verifies their own machine in real time; nothing is copy-pasted, everything is typed
Prerequisites	None — this is the first lab of the semester (marked Midterm-Eligible)
Student-facing lab page	Week 1 In-Class Lab — Module 01, “Developer Mindset & First Setup”
Parts covered	Part 1 (environment verification) – Part 4 (README) + Stretch (Git identity)
Submission	3 screenshots + <code>hello_ism3232.py</code> + <code>README.md</code> , uploaded to Canvas by end of class, completion credit

ISM3232 is structured around a **shell-first developer discipline** from day one — this is not a course that eases into the terminal, it starts there, and every remaining lab this semester assumes students are fluent with what’s built today. The single most consequential moment in this entire lab is the very first line: confirming the terminal prompt shows `%` (zsh), not `$` (bash). Get this wrong and every subsequent command in the semester behaves subtly differently from what the course materials show. Do not let a single student proceed past Task 1 with the wrong shell.

2 Learning Objectives

By the end of this class period, students should be able to:

1. Confirm their terminal is running zsh, not bash, and verify Python 3.10+, Git, and their home directory are all correctly configured.
2. Build a consistent, multi-module course folder structure from the terminal, without using a file browser.
3. Write, save, and run a `.py` file from the terminal, and read the file’s own comment-block questions as prompts for genuine understanding, not just fill-in-the-blank busywork.
4. Write a README documenting what was done and verified — a habit that begins in Week 1 and is required for every remaining lab.

3 Before Class — Setup Checklist

- ☐ Confirm your own demo machine's terminal shows a zsh % prompt before class — if you personally still have a bash default, fix this before demonstrating live, since projecting the wrong prompt on day one undermines the entire lesson.
- ☐ Know your platform's default shell history: **macOS (Catalina and later) defaults to zsh already**; Windows students are most likely using WSL (Windows Subsystem for Linux) or Git Bash configured for zsh per the course's precourse setup — confirm your course's specific Windows setup path before class, since “just open Terminal” doesn't apply uniformly across platforms the way it does for Mac.
- ☐ Walk the room during Task 1 specifically — a wrong shell is the single highest-leverage thing to catch early, and it's much faster to fix one-on-one during Part 1 than to untangle later when a student's `alias/.zshrc` work in Week 3 mysteriously doesn't apply.
- ☐ Have your own `ism3232/` folder structure and `hello_ism3232.py` ready to demo once, live, end to end, before students begin independently.

4 Materials Needed

- A terminal defaulting to zsh, VS Code, Python 3.10+, Git — all per the course's precourse setup
- Students: nothing pre-existing — this lab builds everything from scratch
- A phone or screenshot tool for the three required screenshots

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:05	Welcome: “the terminal is the tool for this entire semester”	5
0:05–0:18	Part 1 — Verify your environment	13
0:18–0:35	Part 2 — Course folder structure	17
0:35–0:58	Part 3 — First Python script + explain-what-happened	23
0:58–1:10	Part 4 — README	12
1:10–1:15	Stretch preview + wrap-up, submission checklist	5

Four required parts fill the class period at a careful, verification-heavy pace appropriate for a first

lab where environment problems are the dominant risk, not conceptual difficulty; the Git-identity Stretch is a genuine two-command task, positioned at the end specifically because Week 4 requires it and this is a low-stakes moment to get it done early.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:05)

Say to the class:

“Welcome to ISM3232 — Business Application Development. This course is built around a professional developer workflow from day one: terminal-first, discipline around verification, and habits — like the README you’ll write today — that repeat every single week. Today has one job: confirm your machine is correctly set up, and build the folder structure everything else this semester lives inside. If anything here doesn’t work, this is the class to raise your hand in — not week 8.”

Do: Write today’s four parts on the board as a visible checklist — Verify, Structure, Script, README — since, like ISM2411’s Week 1, there’s no code output yet to signal progress the way later labs will have.

6.2 Part 1 — Verify Your Environment (0:05–0:18, 13 min)

Teaching goal: Confirm the terminal itself is zsh (not bash), and that Python, Git, and the home directory are all correctly configured — before touching a single line of course content.

Say to the class:

“Open the VS Code integrated terminal. Look at the prompt character right now, before typing anything: if it ends in %, you’re in zsh — correct. If it ends in \$, you’re in bash — stop, raise your hand, don’t run anything else yet. This distinction matters because zsh and bash have real, different behaviors, and every script and alias this course teaches assumes zsh specifically.”

Do, live, narrating each command’s purpose before running it:

```
echo $SHELL          # Expected: /bin/zsh
zsh --version         # Expected: zsh 5.x or higher
python3 --version    # Expected: Python 3.10 or higher
git --version         # Expected: git version 2.x
pwd                  # Expected: your home directory
ls                   # Expected: Desktop Documents Downloads etc.
```

Line-by-line explanation:

- `echo $SHELL` — prints the value of the SHELL environment variable: the shell your terminal is configured to launch. Say explicitly: this is checking *configuration*, not necessarily what’s currently running — worth a one-sentence distinction if a curious student asks, but not something to dwell on today.
- `zsh --version` — confirms zsh itself is actually installed and reports a real version number

— a genuinely different check from the line above, since `$SHELL` could theoretically point somewhere `zsh` isn't actually installed on a broken setup.

- `python3 --version` — same verification as ISM2411's Module 02, now framed as step three of six rather than a standalone check — worth noting the parallel if any students are concurrently in both courses.
- `git --version` — confirms Git is installed; this course reaches Git formally in Week 4, but verifying it now means a broken install surfaces today, not three weeks from now under time pressure.
- `pwd` — should print the student's home directory (e.g., `/Users/yourname` on Mac) — this is the terminal's default starting location when freshly opened, and confirming it now establishes a known baseline before Part 2 builds on top of it.
- `ls` — should show the standard set of top-level folders (Desktop, Documents, Downloads, etc.) — a sanity check that nothing about the home directory itself is unusual or corrupted.

Verified example output (exact values will vary by machine — the pattern, not the specific numbers, is what to check):

```
/bin/zsh
zsh 5.9 (arm64-apple-darwin25.0)
Python 3.12.2
git version 2.51.2
/Users/yourname
Desktop Documents Downloads Library Movies Music Pictures Public
```

Common student mistakes to watch for:

- A `$` prompt instead of `%` — stop this student specifically before they proceed; walking them through changing their default shell (`chsh -s /bin/zsh` on Mac, or the course's specific Windows/WSL guidance) is worth doing now, individually, rather than letting them continue into folder creation on the wrong shell.
- `python3 --version` reporting below 3.10, or command `not` found entirely — route to the precourse setup page rather than attempting a from-scratch install fix live in front of the whole room.
- Confusing this terminal with a **non-integrated** system Terminal app that might have different defaults than VS Code's built-in one — if a student's results look inconsistent with their neighbor's, confirm they're actually using the VS Code integrated terminal, not a separate app.

Check for understanding: “If your `echo $SHELL` had printed `/bin/bash` instead of `/bin/zsh`, what specifically would that mean for the rest of this course?” (Every alias, function, and shell-specific syntax taught starting Week 3 — `.zshrc` configuration specifically — would not apply the same way, or at all, to a bash setup; get a student to state that this isn't a cosmetic preference, it's a real compatibility requirement.)

6.3 Part 2 — Course Folder Structure (0:18–0:35, 17 min)

Teaching goal: Build a complete, multi-module folder structure entirely from the terminal — the physical home for every remaining assignment this semester — and open it properly in VS Code as a **workspace**, not by double-clicking individual files.

Say to the class:

“Eight folders, one command. This structure is going to hold every assignment for the rest of the semester — get the names exactly right now, and you’ll never think about ‘where does this go’ again.”

Live-code this:

```
cd ~
mkdir ism3232
cd ism3232
mkdir module01_setup module02_zsh module03_git_github \
    module04_programming module05_functions \
    module06_oop module07_final_project \
    data screenshots
ls
code .
```

Line-by-line explanation:

- `cd ~` — `~` is a shorthand zsh (and bash) recognizes for “my home directory” — say explicitly this is the same location `pwd` confirmed in Part 1, just referenced by a convenient symbol instead of typing the full path.
- `mkdir ism3232` then `cd ism3232` — create the course’s top-level folder, then move inside it — exactly Module 01’s pattern from ISM2411, if any students are concurrently enrolled, worth a brief cross-course note.
- `mkdir module01_setup module02_zsh ... data screenshots` — **this single command creates eight folders at once**, since `mkdir` accepts multiple names as separate arguments. Point out the backslash (`\`) at the end of two lines: this is a **line continuation** — it tells zsh “this command isn’t finished yet, keep reading the next line as part of the same command” — purely a readability convenience for a long command; typing the entire thing on one physical line would work identically.
- `ls` — confirms all eight folders now exist.
- `code .` — opens the **current directory** (`.`) as a VS Code workspace. Say explicitly, since this is a genuinely important habit the exercise itself calls out: **this is the correct way to start working on a project in VS Code — open the whole folder as a workspace, not double-click one file at a time from a file browser**. Working inside a proper workspace is what makes VS Code’s file explorer, integrated terminal, and Git integration all work together coherently; opening loose individual files loses most of that.

Verify together: the VS Code Explorer panel (left sidebar) should show all eight module/-data/screenshots subfolders nested under `ism3232`.

Common student mistakes to watch for:

- Running the `mkdir` command from the wrong location (not inside `~/ism3232`) — produces eight folders in the wrong place; have students run `pwd` immediately before the `mkdir` line if this happens, and confirm they're exactly where they expect before creating anything.
- Misspelling a folder name (a stray extra underscore, a typo) — not an error, just a folder that doesn't match what future labs expect to find; walk the room checking the Explorer panel against the required eight names exactly, since this is much easier to catch visually now than to discover three weeks from now when a later lab's `cd module02_zsh` fails.
- Using `code filename.py` habitually later in the semester instead of `code .` once per project — not wrong for opening a single file quickly, but worth reinforcing that the *workspace-first* habit established today is the one to default to when starting a new module's work.

Check for understanding: “If you ran `code .` from inside `module01_setup/` instead of from `ism3232/`, what would the VS Code Explorer panel show, and would that be wrong?” (It would show only `module01_setup/`'s own contents as the workspace root, not the whole course structure — not strictly “wrong” for working on that one module in isolation, but a smaller, less useful view than opening the whole `ism3232/` folder; a good check that students understand `code .` opens *whatever the current directory is* as the workspace, not some fixed course-level location.)

6.4 Part 3 — First Python Script (0:35–0:58, 23 min)

Teaching goal: The complete write-save-run loop, in module01_setup/ specifically — plus a genuinely valuable twist on the usual “type it, don’t paste it” instruction: a required comment-block reflection asking students to explain, in their own words, what actually happened when the script ran.

Say to the class:

“First real script. I want you to type every character — not copy-paste — because the muscle memory of typing Python, including the mistakes you’ll inevitably make and fix, is itself part of learning to code. Then, once it runs, you’re going to answer four questions about why it worked, in a comment block. This is not busywork — if you can’t answer these four questions, you don’t yet understand what just happened, even if the script ran.”

Do, live:

```
cd ~/ism3232/module01_setup
touch hello_ism3232.py
code hello_ism3232.py
```

Type this into the file (again: type, do not paste):

```
# hello_ism3232.py
# ISM3232 - Business Application Development
# Author: [Your Name]

print('Hello, ISM3232!')

course_name = 'Business Application Development'
credit_hours = 3
weekly_hours = credit_hours * 2

print(f'Course: {course_name}')
print(f'Expected weekly hours: {weekly_hours}')
print('Environment verified. Week 1 complete.')
```

Line-by-line explanation:

- `touch hello_ism3232.py` — creates an empty file with that exact name (touch is a Unix utility whose original purpose was updating a file’s timestamp; creating a new empty file is a well-known side effect worth mentioning, since the name itself doesn’t obviously suggest “create a file”).
- `code hello_ism3232.py` — opens that specific file in VS Code, inside the already-open workspace from Part 2.

- The header comment block (`# hello_ism3232.py, # ISM3232 - ..., # Author: ...`) — a file-identification convention worth establishing as a habit from the very first script: any reader (an instructor, a future collaborator, or the student themselves in six months) can immediately see what this file is and who wrote it, without reading any code.
- `print('Hello, ISM3232!')` — note the **single quotes**, not double — say explicitly, since ISM2411’s equivalent exercises use double quotes throughout: **Python treats single and double quotes identically for defining a string** — this is purely a style choice, and this course’s materials consistently use single quotes; there’s no functional difference, but consistency within a codebase (matching whatever convention a given project or course uses) is itself a professional habit worth naming.
- `course_name = 'Business Application Development'` — note the **extra spaces before the =**, aligning it with `credit_hours` and `weekly_hours` below — a manual alignment style choice (echoing ISM2411 Module 03’s aligned print labels), purely cosmetic, not required by Python.
- `credit_hours = 3` — an int.
- `weekly_hours = credit_hours * 2` — a derived value, computed from `credit_hours` — worth a quick note that this is the same “compute once, store in a named variable” pattern from every prior arithmetic exercise in any Python course, here modeling a course-planning calculation instead of a business one.
- The three final `print()` calls, including two f-strings — nothing new syntactically if students have prior Python exposure; if this is a genuinely first-ever Python class for some students, treat the f-string substitution briefly the way ISM2411 Module 01 does, without assuming it’s already familiar.

Save (%S / Ctrl+S), then run:

```
python3 hello_ism3232.py
```

Verified output:

```
Hello, ISM3232!
Course: Business Application Development
Expected weekly hours: 6
Environment verified. Week 1 complete.
```

Now, the required reflection — add this comment block below the header, and have every student genuinely answer it, not guess:

```
# --- What happened when I ran this script? ---
# 1. The shell found the python3 interpreter at: ____
# 2. Python read the file from top to bottom: True / False
# 3. The f-string on line 11 evaluated {weekly_hours} to: ____
# 4. The output appeared in: the editor / the terminal / both
```

Model strong answers explicitly, since “fill in the blank” framing can tempt students toward

guessing rather than genuinely checking:

1. **Run which python3 right now, live, and use its actual output** — this is a real, checkable fact about their specific machine, not something to guess at (a good moment to introduce which explicitly: it reports the full path to whatever program a bare command name like python3 actually resolves to).
2. **True** — Python executes a script's statements in the order they appear, top to bottom; this is worth stating as a foundational, provable fact — ask a student “how would you prove this to a skeptic, using this exact file?” (a good answer: reorder two `print()` lines, re-run, and observe the output order changes to match).
3. **6** — `credit_hours * 2` with `credit_hours = 3`.
4. **The terminal** — VS Code's editor pane shows the *source code*; the integrated terminal panel is where `print()`'s actual output appears. This is worth stating explicitly as a real, sometimes-confusing-to-beginners distinction: editing a file and running it produce output in two visually separate places within the same VS Code window.

Common student mistakes to watch for:

- Copy-pasting despite the explicit “type it” instruction — not something you can perfectly police, but worth restating the *why* once more if you notice it happening: typing builds the specific muscle memory of noticing your own typos, which pasting bypasses entirely.
- Answering the reflection questions with guesses rather than checking (e.g., guessing a plausible-looking path for question 1 instead of running `which python3`) — walk the room specifically checking that question 1's answer matches what a live `which python3` on that student's own machine actually reports.
- Running the script from the wrong directory — same category of mistake as every prior course's Module 02, worth a quick `pwd` check if `python3 hello_ism3232.py` reports “No such file or directory.”

Check for understanding: “If I moved this file to `module02_zsh/` but kept running `python3 hello_ism3232.py` from inside `module01_setup/`, what would happen?” (`python3: can't open file 'hello_ism3232.py': No such file or directory` — the exact “wrong directory” failure mode covered at length in comparable courses' terminal modules; a good check that this specific error type is already recognizable even in a brand-new course.)

6.5 Part 4 — README (0:58–1:10, 12 min)

Teaching goal: Write a README documenting the day’s work — the first instance of a documentation habit this course requires every remaining week — including an explicit **AI Use Statement**, worth flagging as a recurring, serious course policy element, not boilerplate.

Say to the class:

“Every lab this semester ends with a README update. Today’s is the simplest version you’ll ever write — treat it as establishing the habit, not just filling in a template. And notice the AI Use Statement at the bottom — this appears in every README template this course provides, and it’s a real, graded honesty requirement, not decoration.”

Do, live:

```
cd ~/ism3232/module01_setup
touch README.md
code README.md
```

Write this content, filling in real details:

```
# ISM3232 - Module 1: Course Setup

**Name:** [Your Name]
**Date:** [Today's Date]

## What I Did
- Verified zsh, Python 3, and Git versions
- Created the ism3232/ course folder structure
- Wrote and ran hello_ism3232.py from the terminal

## Verification Results

| Tool      | Version |
|-----|-----|
| zsh       |         |
| Python 3  |         |
| Git       |         |

## AI Use Statement
I did not use AI for this lab.
```

Line-by-line explanation:

- The Markdown syntax here (#, ##, ****bold****, a pipe-delimited table) is worth a brief explicit note even without deep coverage today: this is the same lightweight formatting language

ISM2411's Module 08 README exercise uses, and it renders automatically, formatted, on GitHub once this repo is pushed (Week 4).

- **The Verification Results table** — have students fill in their *actual* version numbers from Part 1's commands, not placeholder text — this table is a genuine, checkable record, not decoration; a grader (or the student themselves, months later) should be able to look at this table and know exactly what environment this work was verified against.
- **The AI Use Statement** — say explicitly, plainly, and without ambiguity: **this line must be honest**. If a student used an AI tool for any part of today's lab in a way that goes beyond what your course's syllabus permits, the statement needs to reflect that, not default to the template's "I did not use AI" if it isn't true. Frame this as a trust-building habit that starts on day one, not a one-time checkbox — students will write this exact statement, updated as appropriate, in every remaining lab's README this semester.

Common student mistakes to watch for:

- Leaving the version table cells blank rather than filling them with Part 1's actual verified output — walk the room checking this specifically, since it's an easy thing to skip when a README feels like an afterthought after the "real" work of Parts 1–3.
- Copying the AI Use Statement template without genuinely considering whether it's accurate for their own work today — a brief, non-accusatory reminder to the whole room that this is a real statement, not boilerplate, is worth the ten seconds it takes.

Check for understanding: "If you used an AI assistant today only to look up what touch does, without pasting your own code into it, is 'I did not use AI for this lab' still an accurate statement to write?" (This is genuinely a judgment call worth discussing rather than answering definitively — a good answer recognizes that "AI use" likely means something more specific in your course's actual policy than any AI-adjacent activity whatsoever, and that when in doubt, disclosing minor, tool-adjacent use is safer and more honest than omitting it. Point students to your syllabus's specific AI policy for the precise line, since this varies by course and instructor.)

6.6 Stretch — Configure Git Identity (1:10–1:15, as time allows)

Frame as a quick, required-for-later two-command task, not optional depth:

```
git config --global user.name 'Your Name'
git config --global user.email 'you@email.com'
git config --global --list
```

One sentence of framing, worth saying even if time is very short: “This is a one-time setup, and Week 4’s first-ever `git commit` will fail without it — doing it now, while it’s fresh and low-stakes, saves a confusing moment later. `--global` means this identity applies to every Git repository on this machine, not just this course’s, so use your real name and an email you’re comfortable having attached to your commit history — many students use their GitHub account’s email specifically, since that’s what will show up on their public commit history there.”

7 Wrap-Up (last ~5 minutes)

Review the submission checklist together, on screen:

- ☐ **Screenshot 1:** All six verification commands and their output, visible in one screenshot
- ☐ **Screenshot 2:** VS Code Explorer panel showing all eight module subfolders
- ☐ **Screenshot 3:** Terminal showing `hello_ism3232.py`'s output
- ☐ **File upload:** `module01_setup/hello_ism3232.py`, including the answered reflection comment block
- ☐ **File upload:** `module01_setup/README.md`, with name, date, versions, and AI use statement filled in
- ☐ All uploaded to Canvas by end of class

Preview Week 2: “Today verified your tools work. Next week, you actually *use* the terminal to navigate and manipulate files — real practice with the commands that will become completely automatic by the end of this unit.”

8 Appendix A — Full Command & Code Reference

Part 1 (verification):

```
echo $SHELL
zsh --version
python3 --version
git --version
pwd
ls
```

Part 2 (folder structure):

```
cd ~
mkdir ism3232
cd ism3232
mkdir module01_setup module02_zsh module03_git_github \
    module04_programming module05_functions \
    module06_oop module07_final_project \
    data screenshots
ls
code .
```

Part 3 (`module01_setup/hello_ism3232.py`):

```
# hello_ism3232.py
# ISM3232 - Business Application Development
# Author: [Your Name]
```

```

print('Hello, ISM3232!')

course_name    = 'Business Application Development'
credit_hours   = 3
weekly_hours   = credit_hours * 2

print(f'Course: {course_name}')
print(f'Expected weekly hours: {weekly_hours}')
print('Environment verified. Week 1 complete.')

# --- What happened when I ran this script? ---
# 1. The shell found the python3 interpreter at: [output of `which python3`]
# 2. Python read the file from top to bottom: True
# 3. The f-string on line 11 evaluated {weekly_hours} to: 6
# 4. The output appeared in: the terminal

```

Part 4 (module01_setup/README.md): see the full template in the Part 4 walkthrough above.

Stretch (one-time Git identity):

```

git config --global user.name 'Your Name'
git config --global user.email 'you@email.com'
git config --global --list

```

9 Appendix B — Extra Practice (only if the class finishes early)

Four required parts fill the full class period at a normal, verification-careful pace, especially accounting for individual environment troubleshooting. If a section moves unusually fast:

Extra — a second script, independently. Have students create `module01_setup/about_me.py` with at least three variables (name, major, one fact about themselves) and two f-string `print()` statements combining them, run it, and take a screenshot — good extra rehearsal of the exact write-save-run loop, on content they generate themselves rather than typing a provided script.

Extra — explore `ls` variants. Have students try `ls -a` (show hidden files, including any beginning with a dot) and `ls -l` (long format, one file per line with details) separately, before Part 3 of Week 2 introduces `ls -la` as the combined form — a good preview that builds curiosity for next week's lab rather than covering new required content today.